

Navya — Indic-CodecFake

RTX 4060 · you own the matched real/fake Indic data and every cross-dataset number

[Start here](#)

Clean restart, and the answer to your MLAAD question

MLAAD is still yours. Akshat is on IndicSynth — the handbook was reassigned after the first version went out, so the copy you read was stale. Please don't take IndicSynth; you'd be duplicating him and we'd lose a whole synthesis family.

Your 403 is not author approval. MLAAD is *auto-gated*: accept the terms and access is instant, no human in the loop. HuggingFace's own page says “publicly accessible, but you have to accept the conditions”. What almost certainly happened is that your CLI has a valid token but the *account* never clicked accept in a browser — a token alone does not do that. Step 1 fixes it in about two minutes.

YUGAL IS ASLEEP FOR AN HOUR

So this handbook is written to not need him. Step 1 has a decision point: if MLAAD opens up, you carry on with it; if it is still blocked after all three checks, **Step 2B** gives you an hour of genuinely useful work that needs no download and no permission. Do not sit waiting.

Clean restart

```
cd /d/SONIX
git fetch origin
git checkout yugal/ml-pipeline
git pull
```

The pull matters — `augment_folder.py` is new and Step 3 needs it. Confirm:

```
ls prep_mlaad.py augment_folder.py make_indic_spoof.py stamp_labels.py src/extract_embeddings.py
```

All five must list. If `augment_folder.py` is missing you did not pull.

Step 1

Unblock MLAAD — three checks, in order

1. Accept the terms in a browser. Sign in as `navyaanejathegreat`, open `huggingface.co/datasets/mueller91/MLAAD`, and click the button to agree to the conditions. This is the step a CLI token cannot perform, and it is the usual cause of exactly the error you saw.

2. Confirm the CLI is that same account.

```
hf auth whoami
```

If it prints a different username or nothing, run `hf auth login` and paste a token from `huggingface.co/settings/tokens`.

3. Check the token type. A *fine-grained* token needs “Read access to contents of all public gated repos you can access” ticked. A plain **read** token has this by default and is simpler — if in doubt, make a read token and re-login with it.

Then test:

```
python prep_mlaad.py --list
```

DECISION POINT

It printed a list of language folders → go to Step 2A.

Still 403 / “not in the authorized list” → go to Step 2B, and message Yugal what you tried. Do not keep retrying.

Step 2A

MLAAD Indic — the main job

Up to 140 TTS models across 51 languages. The Indic slice is ~45 hours: Hindi 15.1h, Bangla 14.1h, Tamil 5.0h, Kannada 4.2h, Marathi 3.0h, Urdu 2.1h, Malayalam 1.8h. It is from Müller et al. — the same group as In-the-Wild, which we already cite on slide 3.

The 140 models are the point, not the hours. A head trained on one generator learns *that generator*; that is exactly how we ended up with a detector that only understands 2019-era English TTS.

```
python prep_mlaad.py --langs hi,bn,ta,mr,kn,ml,ur `
  --out data\mlaad_indic --per-model 150
```

The full repo is 183 GB; this pulls only the seven language folders, roughly 15 GB, and it resumes if the connection drops — just re-run the identical command.

DO NOT DROP --PER-MODEL

It caps how many clips any single TTS model contributes. Without it one prolific model supplies most of the class and the head learns to detect that model rather than synthesis — a fresh version of the bug we are fixing.

GATE – CHECK THE SPREAD BEFORE SPENDING GPU TIME

The script prints the (language, model) pairs it found. You want **several thousand clips across all seven languages and many models**. One language or one model means something went wrong — stop and tell Yugal rather than extracting a lopsided set.

Then go to Step 3 with `data\mlaad_indic`.

Step 2B

If MLAAD is still blocked — do this instead

No download, no permissions, runs entirely on your 4060. It generates synthetic Indian speech locally with Meta's MMS-TTS — a different TTS system from anything else we have, so it adds real generator diversity rather than being a consolation prize.

```
pip install -U transformers scipy

python make_indic_spoof.py --lang hin --n 500 --out data\indic_spoof
python make_indic_spoof.py --lang mar --n 400 --out data\indic_spoof
python make_indic_spoof.py --lang ben --n 400 --out data\indic_spoof
python make_indic_spoof.py --lang tam --n 400 --out data\indic_spoof
python make_indic_spoof.py --lang tel --n 400 --out data\indic_spoof
```

~2,100 clips. Yugal was going to do ~800 of these; do the larger run on your card instead and tell him, so he does not repeat it. Then continue to Step 3 with `data\indic_spoof` — the rest of the pipeline is identical.

Step 3

Augment the spoofs — do not skip this

This step is new, it is unassigned to anyone else, and skipping it will quietly sink the retrain.

Our Indic *spoofs* (MLAAD, IndicSynth, MMS-TTS) are clean studio-quality synthesis. Our Indic *bonafide* (IndicVoices) is varied real-world recording — phones, rooms, background noise. Train on them as they are and the head learns “**clean Hindi = fake**” instead of “synthetic Hindi = fake”.

That is the same shortcut we already diagnosed once. The original model had learned “clean background = fake” from ASVspoof’s −75 dBFS digital silence, and it flagged genuine recordings above ~38 dB SNR. Your RIR/MUSAN work fixed it for English. If we hand the model clean-vs-noisy split along the label line again, the Bachchan clip fails for a brand-new reason and we will have burned a day.

```
python augment_folder.py --in data\mlaad_indic --out data\mlaad_indic_aug
```

Everything is synthesised in numpy — no RIRS_NOISES, no MUSAN, no ffmpeg needed. It applies reverb (80%), noise at 8–30 dB SNR (80%), and G.711 mu-law (50%). If you have the real RIR/MUSAN unpacked from your earlier work, use them instead:

```
python augment_folder.py --in data\mlaad_indic --out data\mlaad_indic_aug `
--rir-root <RIRS_NOISES> --musan-root <musan>
```

Keep both folders. Clean and augmented both get extracted, both get stamped spoof. The clean ones teach the artefact; the augmented ones stop the channel from becoming the cue.

Step 4

Extract, stamp, verify — for both folders

```
python src\extract_embeddings.py --split train --audio-dir data\mlaad_indic `
--out outputs\embeddings_mlaad_indic --batch 8

python src\extract_embeddings.py --split train --audio-dir data\mlaad_indic_aug `
--out outputs\embeddings_mlaad_indic_aug --batch 8
```

CUDA out of memory → `--batch 4`, then `--batch 2`.

```
python stamp_labels.py --emb-dir outputs\embeddings_mlaad_indic\train --label 1
python stamp_labels.py --emb-dir outputs\embeddings_mlaad_indic_aug\train --label 1

python stamp_labels.py --emb-dir outputs\embeddings_mlaad_indic\train --check
python stamp_labels.py --emb-dir outputs\embeddings_mlaad_indic_aug\train --check
```

GATE – READ BOTH CHECK OUTPUTS

Each must report **every row as spoof (1)**. Any bonafide rows and you stop: training on unstamped fakes teaches the model that synthetic Hindi is genuine, which is strictly worse than not having the data. (This is the tool you fixed on 2 Sept — before that it never once wrote a label successfully.)

Send Yugal

- Both embedding folders
- The (language, model) summary from Step 2
- Both --check outputs, pasted in full
- Which path you took — 2A or 2B

Step 5

Re-measure — once Yugal ships head_v3

Every cross-dataset number has to be re-derived. Nothing on a slide may come from a model we are no longer shipping.

```
python src\eval.py --split eval --emb-root outputs\embeddings_df21 `
  --model-ckpt outputs\models\head_v3.pt --out-scores outputs\scores_df21_v3
python src\metrics.py --split eval --scores-dir outputs\scores_df21_v3 `
  --out-dir outputs\plots_df21_v3

python src\eval.py --split train --emb-root outputs\embeddings_indicvoices_ho `
  --model-ckpt outputs\models\head_v3.pt --out-scores outputs\scores_ho_v3
python flag_rate.py --split train --scores-dir outputs\scores_ho_v3
```

SAY PRECISELY WHAT “FLAGGED” MEANS

flag_rate.py counts one score per 4-second chunk, no smoothing, no hysteresis. The figure is “X% of genuine 4-second chunks”, never “X% of calls” — the demo path applies a 5-window moving average and alarms less often. Conservative is good; imprecise phrasing is what a follow-up question kills.

Also: a **second seed** on the final head, 15 minutes, so “5.27%” becomes a measurement with error bars rather than one number. Until then we say “≈5.3%”. And keep **75% coverage** attached to every DF21 figure.

Housekeeping

- **Back up your embeddings.** DF21 and IndicVoices exist on exactly one machine. If it dies, every cross-dataset number in the deck becomes unreproducible. An external drive is enough.
- **Retire your metrics.py** in favour of Yukti's — hers has --scores-file and log-scale DET axes, and is now merged into yugal/ml-pipeline. Two versions in circulation is how two slides end up disagreeing.

- **Indic-CodecFake** would have been the best dataset for this problem (one-to-one real/fake pairs of the same Indian speaker across 8 neural codecs) but the dataset link on helixometry.github.io/IndicFake is still a placeholder — only the codec code is published. Worth emailing the authors. Do not wait on a reply.

Credit

Why the Indic result was not a wasted day

Your recording-level holdout split is why we can trust any Indic number as generalisation rather than memorisation — it groups by recording id, then recomputes the intersection and *refuses to write* if any recording appears on both sides. That is more care than most published work takes. You also found that `stamp_labels.py` had never once written a label successfully, which was silently corrupting anything anyone stamped.

The IndicVoices result measured a real false-alarm problem and fixed it. It could not catch the second bug, because no Indian spoofs existed anywhere to test against. That is the gap this round closes — and it is only visible *because* your work got us far enough to see it.

Troubleshooting

SYMPTOM	FIX
403 / not in the authorized list	Accept terms in a browser as that account; check <code>hf auth whoami</code> ; use a plain read token
huggingface-cli is deprecated	Use <code>hf</code>
can't open file 'augment_folder.py'	You did not pull
download stops partway	Re-run the identical command — it resumes
none of the requested languages exist	Run <code>--list</code> , use the real folder names
CUDA out of memory	<code>--batch 4</code> , then <code>--batch 2</code>
check shows bonafide rows	Stop. Re-run the stamp; if it persists, tell Yugal
Activate.ps1 cannot be loaded	<code>Set-ExecutionPolicy -Scope Process -Bypass</code>
symlink warning on Windows	Harmless — uses more disk, works fine

When in doubt, stop and ask. A mislabelled dataset is worse than no dataset, and a wrong number is far worse than a missing one.

The situation

What broke, and why

Our detector fails on Indian speech, and we now know exactly why.

Every model up to `head_robust_v2` was trained on ASVspoof 2019 — English studio speech — plus augmentations that vary *rooms, noise and phone channels*. None of them vary *language*. So on 42,082 genuine IndicVoices clips, the baseline called **56% fake** and `robust_v2` called **58.6% fake**. That is why the Modi and Bachchan clips failed.

Navya then added IndicVoices to training and Indian false alarms collapsed to **0.02%**. Real fix? No — a second bug. IndicVoices is **bonafide-only**: every one of its ~37,000 rows is labelled real. So the only thing the model could learn from Indian audio was “*Indian phonetics* → *real*”. Tested on a cloned Amitabh Bachchan clip, `head_full_ho` calls it **real**.

MODEL	REAL BACHCHAN	CLONED BACHCHAN	WHAT IT ACTUALLY LEARNED
head_robust_v2	fake ✗	fake ✓	“Indian audio = fake”
head_full_ho	real ✓	real ✗	“Indian audio = real”

Neither is detecting anything. Each gets one column right by accident.

THE ROOT CAUSE, IN ONE SENTENCE

As long as every Indian-language sample in training is labelled real, the language itself is a shortcut to “real”, and a cloned Indian voice walks straight through. No threshold, augmentation or calibration fixes this.

The fix

Put Indian languages on **both sides of the label**, from **several different synthesis families**. Then the model cannot use language or accent as a cue — it has to find real synthesis artefacts or nothing.

DATASET	SYNTHESIS FAMILY	OWNER
Indic-CodecFake	Neural audio codecs — 8 of them (DAC, Encodec, SoundStream, SpeechTokenizer, FunCodec, AudioDec, SNAC, MIMI)	Navya
MLAAD (Indic slice)	Text-to-speech — up to 140 models	Akshat
IndicSynth	Voice conversion (freevc24)	Akshat
IndicVoices <code>have it</code>	Real human speech (bonafide side)	Navya

Three *families*, not just more hours. A head trained on one generator learns that generator — which is precisely the trap we fell into with English-only ASVspoof 2019. Generator diversity is what buys generalisation.

How we train

One model, many ablations

Two questions came up, so here they are answered for everyone.

Is the new model built from scratch, or on top of the latest one?

Neither, exactly — and this matters. Our pipeline has two stages:

STAGE	COST	REUSABLE?
audio → embeddings (frozen wav2vec2 XLS-R, 300M params)	GPU, hours	Yes — forever, per dataset
embeddings → head (MLP, 262,657 params)	CPU/GPU, minutes	Cheap to redo

So we never “continue training” `head_full_ho`. We retrain a **fresh head on the union of every embedding root** — the old ones plus the new ones. That is what `--extra-emb-root` does, and it stacks (it uses `action="append"`, verified).

Fine-tuning the existing head on only the new data would be actively wrong: it would forget the English attacks it can currently catch.

One model or several?

One production model, plus four throwaway ablation heads for the results slide. Once the embeddings exist, each head takes minutes, so the ablations are nearly free — and they are what let us say “*we measured which data helped*” instead of “*we threw everything in and hoped*”.

The expensive, parallel part is extraction. That is why it is split three ways across our GPUs. Training is serial and Yugal does it.

THE RULE THAT MAKES OR BREAKS THIS

Cap clips per (language × generator). If one TTS model contributes 40,000 clips and another contributes 200, the head learns to detect the loud one. Every extraction step below has a cap flag. Use it.

Hard rules

What must never enter training

These are test sets. Training on any of them destroys the numbers that prove we generalise, and a judge who spotted it would be right to dismiss the project.

- **ASVspoof 2021 DF** — our cross-dataset headline
- **In-the-Wild** — the honest generalisation test, named on slide 3
- **The IndicVoices holdout** (`embeddings_indicvoices_ho`) — 12% held out by recording
- **Akshat's 20 labelled clips** and **the Bachchan pair** — our real-world evidence

Everything else — ASVspoof 2019 train, the augmented copies, IndicVoices train split, Indic-CodecFake, MLAAD, IndicSynth — is fair training data.

SECOND HARD RULE – LABEL STAMPING

`extract_embeddings.py --audio-dir` marks **everything label 0 (real)** by default. Every spoof folder must be stamped `--label 1` afterwards, and verified with `--check`. Skip it and your fakes train as genuine — a silent, catastrophic bug that nothing warns you about.

`stamp_labels.py` was broken until 2 Sept (it never wrote a label successfully). Make sure you have pulled since then, and always read the `--check` output rather than assuming.

Sections after your own tasks are identical in all three handbooks, so nobody has to read someone else’s. Datasets: Indic-CodecFake (helixometry.github.io/IndicFake) · MLAAD (deepfake-total.com/mlaad) · IndicSynth (huggingface.co/datasets/vdivyasharma/IndicSynth). When in doubt, stop and ask Yugal — a wrong number is far worse than a missing one.